# Trabajo Práctico N.º 8: Food Store en la Feria

La base de la tablet, sincronización con semáforo y cierre integral de la capa de datos

## 1. Consigna 1.a · Matriz de Transferencia de Datos hacia la Tablet Móvil

En un esquema *Offline-First* donde la tablet del Food Truck opera sin conexión confiable a Internet en la feria, se diseñó una instantánea mínima y desensibilizada. Solo viajan las columnas estrictamente requeridas para la toma de pedidos, impidiendo la exposición de datos sensibles o estratégicos ante eventuales robos o pérdidas físicas del dispositivo.

| Columna | ¿Viaja? | Justificación Funcional y de Seguridad |
|---|---|---|
| **id** | **SÍ** | Identificador numérico indispensable para relacionar los ítems del borrador con el catálogo descargado. |
| **nombre** | **SÍ** | Denominación visible en la interfaz para que el mozo identifique el producto en mostrador. |
| **precio_centavos** | **SÍ** | Precio de lista convertido a entero en centavos para calcular totales preliminares sin error de flotante. |
| **stock_visto** | **SÍ** | Existencia física al momento de la instantánea para alertar tempranamente si el producto ya estaba agotado. |
| **disponible (activo)** | **SÍ** | Bandera booleana (0/1) que habilita o deshabilita la selección del producto en el menú móvil. |
| **eliminado (lápida)** | **SÍ** | Mantiene integridad referencial local pero oculta el producto del catálogo comercial activo. |
| **fecha_copia** | **SÍ** | Marca temporal que permite a la aplicación móvil auditar la antigüedad de su caché local. |
| **Datos de usuarios (hash/mail)** | **NO** | Datos personales y credenciales sensibles (PII). No se requieren para armar borradores de venta. |
| **Costos de compra / márgenes** | **NO** | Secreto comercial del negocio. Si la tablet es sustraída en la feria, no se filtran costos ni proveedores. |
| **Historial de pedidos de otras tiendas** | **NO** | Saturaría el almacenamiento de la tablet y violaría el principio de mínimo privilegio. |

***Regla de Oro: El móvil propone, el servidor dispone.***
*PostgreSQL es la única Fuente de Verdad (Single Source of Truth). La tablet jamás confirma ventas fiscales ni altera stock; el estado SIMULADO_APROBADO es una pre-validación teórica, no una venta real.*

## 2. Consigna 2.c · Resolución de Casos con el Semáforo de Sincronización

El semáforo evalúa atómicamente cada solicitud en estado PENDIENTE contra snapshot_servidor respetando la jerarquía obligatoria: **Rojo (RECHAZADO) > Amarillo (CONFLICTO) > Verde (SIMULADO_APROBADO)**.

| Caso | Preparación y Comandos | Resultado Observado | Explicación Técnica y Justificación |
|---|---|---|---|
| **C2**<br>**Subida de precio y aceptación** | `1. Borrador con Muzza a $100.00 (10000 centavos).`<br>`2. Servidor sube precio a $125.00.`<br>`3. Cliente acepta: se actualiza borrador a 12500 y se emite nueva solicitud.` | Paso 1: **CONFLICTO**<br>Paso 2: **SIMULADO_APROBADO** | El semáforo detecta discrepancia de precio sin problemas de stock. Al aceptar el cliente, se reintenta y aprueba limpiamente. |
| **C3**<br>**Quiebre de stock** | `1. Borrador pidiendo 50 Napolitanas.`<br>`2. Servidor tiene stock = 10.` | **RECHAZADO**<br>(Stock insuficiente) | La condición cantidad > stock activa inmediatamente la prioridad Roja, impidiendo sobreventas en la feria. |
| **C4**<br>**Baja lógica (lápida)** | `1. Borrador con Producto 3 activo.`<br>`2. Servidor marca eliminado = 't'.`<br>`3. Recarga de catálogo en la tablet.` | **RECHAZADO**<br>(Lápida activa) | El producto dado de baja no se puede vender. La fila persiste en SQLite (integridad referencial) pero se oculta del catálogo activo. |
| **C5**<br>**Aprobación limpia** | `1. Borrador con Producto 4 (2 un. a $4138.00). Coincide exactamente con el servidor.` | **SIMULADO_APROBADO** | Hay stock, el precio es idéntico y el producto está activo. Queda listo para su posterior confirmación transaccional. |
| **C6**<br>**Reenvío / Idempotencia** | `1. Reenvío con el mismo solicitud_uuid de C5.`<br>`2. Re-ejecución del semáforo.` | **Error PRIMARY KEY**<br>(Estado C5 invariante) | La clave primaria rechaza duplicados locales. El semáforo ignora solicitudes no pendientes, garantizando idempotencia. |

**Protección real en sistema de producción (Parada 8):** En una arquitectura real, la protección de idempotencia reside en la API del backend mediante encabezados HTTP (Idempotency-Key: <UUID>) y una tabla central con restricción UNIQUE. Si la red se corta y el móvil reintenta, el servidor reconoce la clave y retorna la respuesta original sin volver a descontar stock ni duplicar cobros.

**Comprobación de Invariabilidad en PostgreSQL:** Tras correr todas las simulaciones de la tablet, se ejecutó:
SELECT COUNT(*) FROM pedido; → **200,001 pedidos** (0 ventas insertadas).
SELECT id_producto, stock FROM producto WHERE id_producto IN (1,2,3,4); → Muzza: **5**, Napolitana: **10**, Prod. 3: **115**, Prod. 4: **130** (0 unidades descontadas).
*Esto comprueba empíricamente que la operatoria móvil no altera la base transaccional central.*

## 3. Consigna 3.a · Mapa de la Capa de Datos del Ecosistema Food Store

A lo largo de la cursada integramos tres motores complementarios, asignando a cada uno una responsabilidad arquitectónica precisa:

| Motor de Base de Datos | Rol y Patrón Arquitectónico | ¿Qué datos guarda? | ¿Quién manda sobre cada dato? (Autoridad) |
|---|---|---|---|
| **PostgreSQL 18 (Servidor Central)** | **Núcleo Transaccional (OLTP)** Single Source of Truth (SSoT) Consistencia ACID estricta | • Usuarios, contraseñas y roles (RBAC).<br>• Catálogo maestro de productos y precios oficiales.<br>• Stock físico real e inventario.<br>• Pedidos confirmados y facturación fiscal.<br>• Auditoría forense y logs de conexiones. | **AUTORIDAD ABSOLUTA** sobre precios, stock, ventas fiscales y usuarios. Ningún otro motor puede modificar estos datos directamente. |
| **MongoDB (Unidad 7)** | **Read Model Documental** Arquitectura CQRS Consultas masivas web | • Pedidos desnormalizados con líneas de productos embebidas (lineas).<br>• Instantáneas de clientes embebidos (cliente).<br>• Catálogo JSON optimizado para APIs web. | **AUTORIDAD SUBORDINADA (Lectura).** Si ocurre una discrepancia, PostgreSQL manda y la colección de MongoDB se regenera desde el origen. |
| **SQLite 3.50+ (Unidad 8)** | **Base Embebida de Borde (Edge)** Tablet Móvil del Food Truck Operatoria Offline-First | • Instantánea descargada (snapshot_servidor).<br>• Catálogo en caché (producto_cache).<br>• Borradores de pedidos tomados en la feria.<br>• Ítems temporales y cola de sincronización. | **AUTORIDAD LOCAL LIMITADA** a la interfaz del cajero y borradores locales. **CERO autoridad** sobre stock y precios del servidor central. |

## 4. Consigna 3.b · Plan de Mantenimiento y Recuperación Integral

Como extensión directa del plan de contingencia probado en la Unidad 6 (RTO de 4.84 s y RPO de 1.70 min con restore_test.ps1):

| Aspecto Operativo | Estrategia y Procedimiento de Mantenimiento |
|---|---|
| **¿Qué se respalda y cómo se prueba la restauración?** | • **PostgreSQL:** Se respalda diariamente con pg_dump comprimido y archivado continuo de WALs (Point-In-Time Recovery). La restauración se valida de forma automática mediante restore_test.ps1 en la base efímera foodstore_restore_test verificando la integridad de tablas y restricciones.<br>• **MongoDB:** Backups semanales con mongodump. Si se corrompe, se regenera desde el catálogo relacional.<br>• **SQLite (Tablets):** Las bases locales **NO** se respaldan. Su arquitectura es descartable (*ephemeral*): si una tablet se rompe o extravía, se provisiona una nueva, se descarga la instantánea fresca y se continúa operando. Solo se resguardan temporalmente los borradores no sincronizados vía USB o red local si la conexión falla. |
| **¿Qué hacen las tablets ante un restore viejo de PostgreSQL?** | Cada instantánea exportada incluye una marca de época (snapshot_epoch). Si el servidor sufre un incidente y restaura un backup de 24 horas atrás, la tablet detecta que el epoch del servidor es menor al de su catálogo local. La tablet **invalida inmediatamente su caché local (producto_cache)**, bloquea la emisión de nuevos pedidos y pasa todas las solicitudes no sincronizadas a estado CONFLICTO_RESTAURACION para su re-evaluación en mostrador, previniendo sobreventas o tarifas desfasadas. |
| **¿Cada cuánto y quién revisa las solicitudes en CONFLICTO o RECHAZADO?** | • **En tiempo real durante la feria:** El mozo/cajero que opera la tablet. Si da CONFLICTO (cambio de precio), consulta al cliente en mostrador si acepta la diferencia; si da RECHAZADO (sin stock), cancela el ítem de inmediato.<br>• **Cierre de jornada (cada 24 horas):** El encargado de feria y el administrador de datos (admin_datos) auditan la tabla cola_sync para detectar quiebres sistemáticos de stock, productos descatalogados o fallas de conectividad. |

## 5. Consigna 3.c · Code Review con IA sobre Scripts de la Cursada

Se solicitó a la herramienta de IA una revisión técnica de los scripts desarrollados en las Unidades 5, 6 y 8. A continuación se detallan los hallazgos con su evaluación crítica y la comprobación empírica en las bases:

| ID / Unidad | Componente | Propuesta de la IA | Decisión Humana | Motivo Técnico y Comprobación en Base |
|---|---|---|---|---|
| **H-1 Unidad 5** | fn_autenticar (Seguridad) | Cambiar la función a SECURITY DEFINER con search_path seguro. | **ACEPTADO Y CORREGIDO** | Como SECURITY INVOKER, el usuario app_web (sin SELECT sobre usuario) recibía permiso denegado. Se corrigió a SECURITY DEFINER asignando el owner a admin_datos. |
| **H-2 Unidad 6** | maintenance.sql (Optimización) | Ejecutar VACUUM FULL ANALYZE diariamente para compactar espacio. | **DESCARTADO CON FUNDAMENTO** | VACUUM FULL adquiere bloqueo exclusivo (ACCESS EXCLUSIVE LOCK), impidiendo lecturas y escrituras durante minutos y destruyendo la disponibilidad de la tienda en producción. |
| **H-3 Unidad 8** | schema_sqlite.sql (Móvil) | Usar claves autonuméricas INTEGER PRIMARY KEY en borradores locales. | **DESCARTADO CON FUNDAMENTO** | En un entorno distribuido offline con múltiples tablets, los enteros autoincrementales colisionan inevitablemente al sincronizar. Se reemplazó por **UUID v4**. |

**Desarrollo y Comprobación del Descarte Crítico (H-2 · VACUUM FULL):**
La IA propuso VACUUM FULL argumentando que recupera espacio en disco tras borrados masivos. Sin embargo, al verificar en la documentación de PostgreSQL y en el catálogo pg_locks, VACUUM FULL reescribe físicamente la tabla adquiriendo un AccessExclusiveLock. Esto bloquea absolutamente todas las consultas SELECT de los clientes en la tienda web y las transacciones de las tablets. En una base con 200,000 pedidos, esta operación degrada el SLA y paraliza la facturación. El descarte se fundamentó manteniendo el mantenimiento estándar con VACUUM (ANALYZE, VERBOSE), el cual limpia tuplas muertas de forma concurrente sin bloquear lecturas ni escrituras.